병렬화: 병렬 처리 최적화

개요

ZKEVM 구현체는 증명 생성 과정에서 병렬 처리를 적극 활용하여 성능을 향상시킵니다. Rust의 Rayon 라이브러리를 사용한 데이터 병렬 처리와 Halo2의 `parallel_syn` 기능을 통한 회로 합성 병렬화가 핵심입니다.

병렬화는 다음 영역에서 적용됩니다:

1. **회로 합성(Circuit Synthesis)**: Witness 할당을 병렬로 수행
2. **데이터 처리**: Witness 생성 시 데이터 변환을 병렬로 처리
3. **테스트 실행**: 대량의 테스트 케이스를 병렬로 실행
4. **증명 생성**: 여러 청크의 증명을 병렬로 생성

parallel_syn 기능

기능 활성화

`parallel_syn` 은 Cargo feature로 제공되며, 기본적으로 활성화되어 있습니다.

파일: `zkevm-circuits/Cargo.toml`

```
[features]
default = ["test", "test-circuits", "debug-annotations", "parallel_syn", "halo2_
parallel_syn = ["hash-circuit/parallel_syn", "halo2_proofs/parallel_syn", "mpt-c
```

이 기능은 다음 크레이트에 전파됩니다:

- `halo2_proofs` : Halo2 백엔드의 병렬 합성 지원
- `hash-circuit` : Keccak 등 해시 회로의 병렬 처리
- `mpt-circuits` : MPT 회로의 병렬 할당

의존성 전파

파일: `prover/Cargo.toml`

```
[features]
parallel_syn = ["halo2_proofs/parallel_syn", "zkevm-circuits/parallel_syn"]
```

파일: `testool/Cargo.toml`

```
[features]
parallel_syn = ["halo2_proofs/parallel_syn", "zkevm-circuits/parallel_syn", "pro
inner-prove = ["prover/test", "parallel_syn", "scroll"]
chunk-prove = ["prover/test", "parallel_syn", "scroll"]
```

State Circuit 병렬 할당

State Circuit은 가장 복잡한 병렬 할당 구현을 가지고 있습니다.

병렬 할당 활성화

파일: `zkevm-circuits/src/state_circuit.rs`

```
#[cfg(feature = "parallel_syn")]
```

```
{
// parallel_syn 기능이 활성화되면 기본적으로 병렬 할당 사용
// 환경 변수로 순차 할당으로 전환 가능:
// STATE_CIRCUIT_ASSIGNMENT_TYPE=serial
let  assignment_type = std::env::var("STATE_CIRCUIT_ASSIGNMENT_TYPE")
.ok()
.unwrap_or_default();
let  is_parallel_assignment = match  assignment_type.as_str() {
"serial" => false,
"parallel" => true,
&_ => true, // 기본값: 병렬
};
if  is_parallel_assignment {
config.assign_par(
layouter,
&self.rows,
self.n_rows,
&self.updates,
randomness,
#[cfg(any(feature = "test", test, feature = "test-circuits"))]
&self.overrides,
&self.exports,
)?;
```

```
} else {
// 순차 할당
config.assign(/* ... */)?;
}
}
```

병렬 할당 구현

파일: zkevm-circuits/src/state_circuit.rs

```
#[cfg(feature = "parallel_syn")]
fn assign_par(
&self,
layouter: &mut impl Layouter<F>,
rows: &[Rw],
n_rows: usize,
updates: &MptUpdates,
randomness: Value<F>,
// ...
) -> Result<(), Error> {
let (rows, padding_length) = RwMap::table_assignments_prepad(rows, n_rows);
let rows_len = rows.len();
// 1. 사용 가능한 스레드 수 확인
let num_threads = std::thread::available_parallelism()
```

```
.map(|e| e.get())
.unwrap_or(1);
// 2. 청크 크기 계산
let chunk_size = (rows_len + num_threads - 1) / num_threads;
let chunk_num = (rows_len + chunk_size - 1) / chunk_size;
log::debug!(
"rows.len() = {}, num_threads = {}, chunk_size = {}",
rows_len, num_threads, chunk_size
);
// 3. Phase 1: RW 테이블 병렬 할당
layouter.assign_regions(
|| "state circuit (synthesize_sub) part1",
rows.chunks(chunk_size)
.zip(is_first_time_vec.iter_mut())
.map(|(rows, is_first_time)| {
move |mut region: Region<'_, F>| {
if *is_first_time {
*is_first_time = false;
// 첫 번째 패스: region shape 획득
region.assign_advice(/* ... */)?;
return Ok(());
}
// RW 테이블 부분 할당
```

```
self.rw_table.load_with_region_part(
&mut region, rows, randomness
)?;
Ok(())
}
})
.collect_vec(),
)?;
// 4. Phase 2: Sort keys 및 제약 조건 병렬 할당
let is_first_access_chunks = layouter.assign_regions(
|| "state circuit (synthesize_sub) part2",
indices.chunks(chunk_size)
.zip(is_first_time_vec.iter_mut())
.enumerate()
.map(|(part_idx, (indices, is_first_time))| {
let rows = rows.as_slice();
move |mut region: Region<'_, F>| {
if *is_first_time {
*is_first_time = false;
region.assign_advice(/* ... */)?;
return Ok(vec![]);
}
log::debug!(
```

```
"[part {}] state circuit: {} ~ {}",
part_idx,
indices.first().unwrap(),
indices.last().unwrap()
);
self.assign_with_region_part(
&mut region, rows, indices, updates, randomness
)
}
})
.collect::<Vec<_>>(),
)?;
// 5. Phase 3: State roots 순차 할당
// TODO: 이 부분도 병렬화 가능
layouter.assign_region(
|| "state circuit (synthesize_sub) part3",
|mut region| {
self.assign_state_roots(
&mut region,
&rows,
padding_length,
&is_first_access_vec,
updates,
```

```
randomness,
)
},
)
}
```

병렬 할당 전략

State Circuit의 병렬 할당은 3단계로 구성됩니다:

1. **Part 1: RW 테이블 할당**

- 전체 행을 청크로 분할
- 각 청크를 독립적인 region에 병렬 할당
- 각 스레드가 자신의 청크만 처리

2. **Part 2: Sort keys 및 제약 조건 할당**

- 인덱스를 청크로 분할
- 각 청크에 대해 병렬로 제약 조건 할당
- 이전 행 참조가 필요하므로 전체 rows 배열 공유

3. **Part 3: State roots 할당**

- 현재는 순차 처리
- 향후 병렬화 가능 (TODO 주석 참조)

MPT Circuit 병렬 할당

MPT Circuit은 환경 변수로 병렬/순차 할당을 제어합니다.

파일: zkevm-circuits/src/mpt_circuit.rs

```
fn synthesize_sub(
&self,
config: &Self::Config,
challenges: &Challenges<Value<Fr>>,
layouter: &mut impl Layouter<Fr>,
) -> Result<(), Error> {
config.0.assign(layouter, &self.proofs, self.row_limit)?;
// 기본적으로 병렬 할당 사용
// PARALLEL_SYN=false로 순차 할당 전환 가능
let use_seq = std::env::var("PARALLEL_SYN")
.map_or(false, |s| s == *"false");
if !use_seq {
// 병렬 할당
config.1.load_par(
layouter,
&self.mpt_updates,
self.row_limit,
challenges.evm_word(),
)?;
} else {
// 순차 할당
```

```
config.1.load(
layouter,
&self.mpt_updates,
self.row_limit,
challenges.evm_word(),
)?;
}
Ok(())
}
```

Rayon을 통한 데이터 병렬 처리

Keccak Circuit의 병렬 처리

Keccak Circuit은 여러 해시 입력을 병렬로 처리합니다.

파일: `zkevm-circuits/src/keccak_circuit/keccak_packed_multi.rs`

```
use rayon::{iter::IntoParallelRefIterator, prelude::ParallelIterator};

// 여러 바이트 배열을 병렬로 Keccak 행으로 변환
let  real_rows: Vec<_> = bytes
.par_iter() // Rayon 병렬 반복자
.flat_map_iter(|bytes| keccak_rows(bytes, challenges))
.collect();
```

동작 원리:

1. `par_iter()` : 병렬 반복자 생성
2. `flat_map_iter()` : 각 입력을 병렬로 처리하고 결과를 평탄화
3. `collect()` : 모든 결과를 하나의 벡터로 수집

Witness 생성 병렬 처리

파일: `zkevm-circuits/src/witness/rw.rs`

```rust
use rayon::prelude::{ParallelBridge, ParallelIterator};


let  errs = rows
.into_values()
.par_bridge() // 순차 반복자를 병렬 반복자로 변환
.flat_map(|group| {
// 각 그룹을 병렬로 검증
group.into_iter().filter_map(|row| {
// 검증 로직
})
})
.collect::<Vec<_>>();
```

테스트 병렬 실행

파일: `testool/src/statetest/suite.rs`

```
use rayon::prelude::*;


// 테스트 케이스를 그룹으로 나누고 병렬 실행
groups
.into_par_iter()
.for_each(|chunk| {
chunk.into_iter().for_each(|ref  tc| {
run_state_test(tc)
})
});
```

파일: `testool/src/bin/trace-gen.rs`

```
use rayon::prelude::*;


// 상태 테스트를 병렬로 처리하여 트레이스 생성
state_tests.into_par_iter().for_each(|st| {
let  error_report = error_report.clone();
let  id = st.id.clone();
// 트레이스 생성 로직
});
```

산술 연산 테스트 병렬화

파일: `bus-mapping/src/evm/opcodes/arithmetic.rs`

```rust
use rayon::iter::{IntoParallelIterator, ParallelIterator};


// 10,000개의 무작위 입력을 병렬로 테스트
(0..10000).into_par_iter().for_each(|_| {
let  inputs: [Word; N_POPS] = (0..N_POPS)
.map(|_| U256(thread_rng().gen::<[u64; 4]>()))
.collect::<Vec<_>>()
.try_into()
.unwrap();
test_handle(inputs, expected);
});
```

환경 변수 제어

STATE_CIRCUIT_ASSIGNMENT_TYPE

State Circuit의 할당 방식을 제어합니다.

```
# 병렬 할당 (기본값)
STATE_CIRCUIT_ASSIGNMENT_TYPE=parallel


# 순차 할당
```

```
STATE_CIRCUIT_ASSIGNMENT_TYPE=serial
```

PARALLEL_SYN

MPT Circuit의 할당 방식을 제어합니다.

```
# 병렬 할당 (기본값)
# 환경 변수 미설정 또는 다른 값


# 순차 할당
PARALLEL_SYN=false
```

성능 향상 측정

병렬화의 이점

1. **CPU 활용도 향상**

- 멀티코어 CPU의 모든 코어 활용
- `std::thread::available_parallelism()` 로 최적 스레드 수 결정

2. **처리 시간 단축**

- State Circuit: 대량의 RW 연산을 병렬 처리
- Keccak Circuit: 여러 해시 입력을 동시 처리
- 테스트: 수천 개의 테스트 케이스를 병렬 실행

3. **메모리 효율성**

- 각 스레드가 독립적인 청크 처리

- 공유 데이터는 읽기 전용으로 사용

성능 측정 예시

```
use std::time::Instant;

let start = Instant::now();

// 병렬 처리
let result = data.par_iter()
.map(|item| process(item))
.collect::<Vec<_>>();

let duration = start.elapsed();
log::info!("Parallel processing took: {:?}", duration);
```

CI/CD에서의 병렬 테스트

파일: `.github/workflows/ci.yml`

```
- name: Run parallel assignment tests(bytecode)
run: cargo test --release --package zkevm-circuits --lib bytecode_circuit::test

- name: Run parallel assignment tests(state)
```

```
run: cargo test --release --package zkevm-circuits --lib state_circuit::test --f
```

병렬화 모범 사례

1. 데이터 분할

```
// 데이터를 균등하게 분할
let chunk_size = (total_items + num_threads - 1) / num_threads;
let chunks = data.chunks(chunk_size);
```

2. 독립적인 작업 단위

```
// 각 작업이 독립적이어야 함
items.par_iter()
.map(|item| {
// item만 사용, 외부 상태 변경 없음
process_independently(item)
})
.collect()
```

3. 공유 데이터는 읽기 전용

```
// 전체 데이터는 읽기 전용으로 공유
```

```
let rows = rows.as_slice();
indices.chunks(chunk_size)
.map(|indices| {
move |mut region| {
// rows는 읽기만 수행
self.assign_with_region_part(&mut region, rows, indices, ...)
}
})
```

4. 결과 수집

```
// 병렬 처리 결과를 효율적으로 수집
let results: Vec<_> = data
.par_iter()
.map(|item| process(item))
.collect(); // 자동으로 동기화
```

제한사항 및 고려사항

1. 순차 의존성

일부 작업은 순차적으로 수행해야 합니다:

```
// State roots 할당은 현재 순차 처리
// TODO: 향후 병렬화 가능
```

```
layouter.assign_region(
|| "state circuit (synthesize_sub) part3",
|mut region| {
self.assign_state_roots(/* ... */)
},
)
```

2. 메모리 오버헤드

병렬 처리는 추가 메모리를 사용합니다:

- 각 스레드의 스택 공간
- 중간 결과 저장
- 동기화 오버헤드

3. 스레드 수 조정

```
// 시스템의 물리적 코어 수 사용
let num_threads = std::thread::available_parallelism()
.map(|e| e.get())
.unwrap_or(1); // 실패 시 단일 스레드
```

4. 디버깅 어려움

병렬 코드는 디버깅이 어려울 수 있습니다:

- 비결정적 실행 순서

- 경쟁 조건(race condition) 가능성
- 로그 메시지 순서 불일치

관련 문서

- CellManager 최적화: 셀 할당 최적화
- RLC 기법: 데이터 압축 최적화
- Lookup 최적화: 룩업 테이블 최적화
- State Circuit: State Circuit 상세
- 증명 집계: 청킹 및 집계 전략

참고 자료

- Rayon 문서: Rust 병렬 처리 라이브러리
- Halo2 parallel_syn: Halo2 병렬 합성 기능
- Rust 표준 라이브러리: `std::thread::available_parallelism()`